

LAPORAN PRAKTIKUM APLIKASI BERBASIS PLATFORM

PERTEMUAN 8
RUNNING MODUL, PENGENALAN FLUTTER, PENGENALAN DART



Disusun Oleh :

Muhammad Dhimas Hafizh Fathurrahman
2311102151
PS1IF-11-REG04

Dosen Pengampu :

Cahyo Prihantoro, S.Kom., M.Eng

Asisten Praktikum :

Gilang Saputra
Rangga Pradarrell Fathi

LABORATORIUM HIGH PERFORMANCE
PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM PURWOKERTO
2026

LINK GIT

https://github.com/Masdim37/2311102151_M-Dhimas-Hafizh-F_Repository-ABP/tree/master/Pertemuan-8

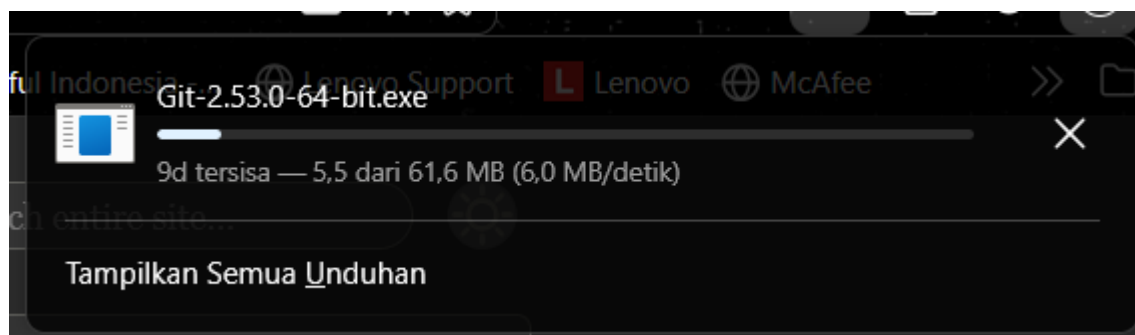
MODUL 1 - RUNNING MODUL

GIT

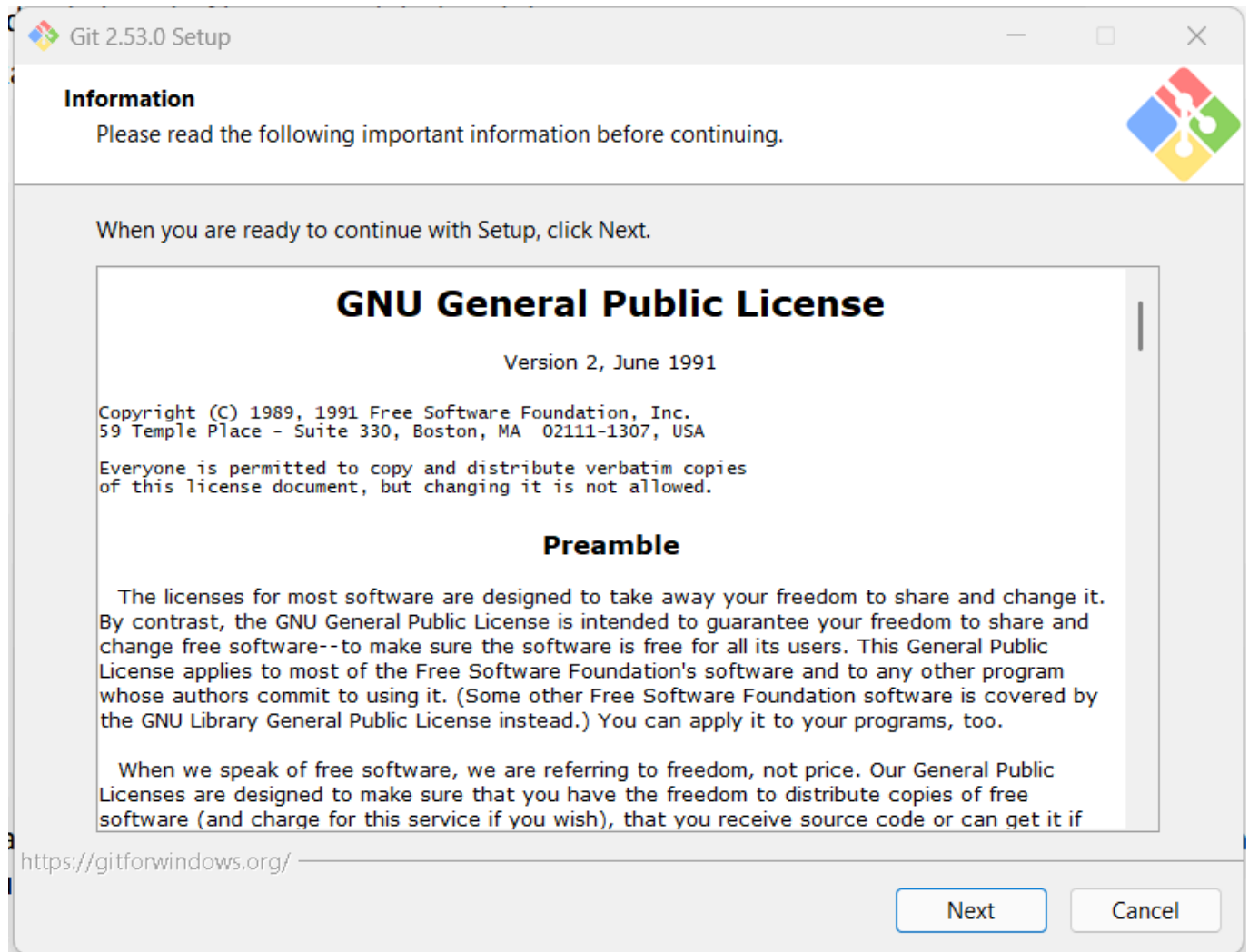
Git adalah salah satu sistem pengontrol versi (Version Control System / VCS) yang sangat populer dalam pengembangan perangkat lunak, diciptakan oleh Linus Torvalds pada tahun 2005. Git digunakan untuk melacak perubahan pada file, terutama dalam proyek pengembangan software, sehingga setiap perubahan yang dilakukan dapat dicatat, dibandingkan, dan jika diperlukan dapat dikembalikan ke versi sebelumnya. Pengontrol versi memiliki peran penting dalam proses pengembangan, baik yang dilakukan secara individu maupun tim. Dengan Git, setiap perubahan pada file proyek akan tersimpan dalam bentuk riwayat (history). Hal ini memungkinkan developer untuk mengetahui siapa yang mengubah apa, kapan perubahan dilakukan, serta alasan di balik perubahan tersebut. Selain itu, Git juga memudahkan dalam mengelola versi berbeda dari suatu proyek, misalnya ketika mengembangkan fitur baru tanpa mengganggu versi utama yang stabil. Git dikenal sebagai distributed revision control system (DVCS), yang berarti setiap developer memiliki salinan lengkap (clone) dari seluruh repository, termasuk seluruh riwayat perubahannya. Berbeda dengan sistem terpusat (centralized VCS) seperti Subversion (SVN), yang hanya menyimpan repository utama di satu server, Git memungkinkan setiap pengguna bekerja secara offline karena seluruh data tersedia secara lokal. Sinkronisasi dengan repository pusat (remote repository) dapat dilakukan kapan saja melalui proses push dan pull.

Berikut langkah-langkah instalasi Git:

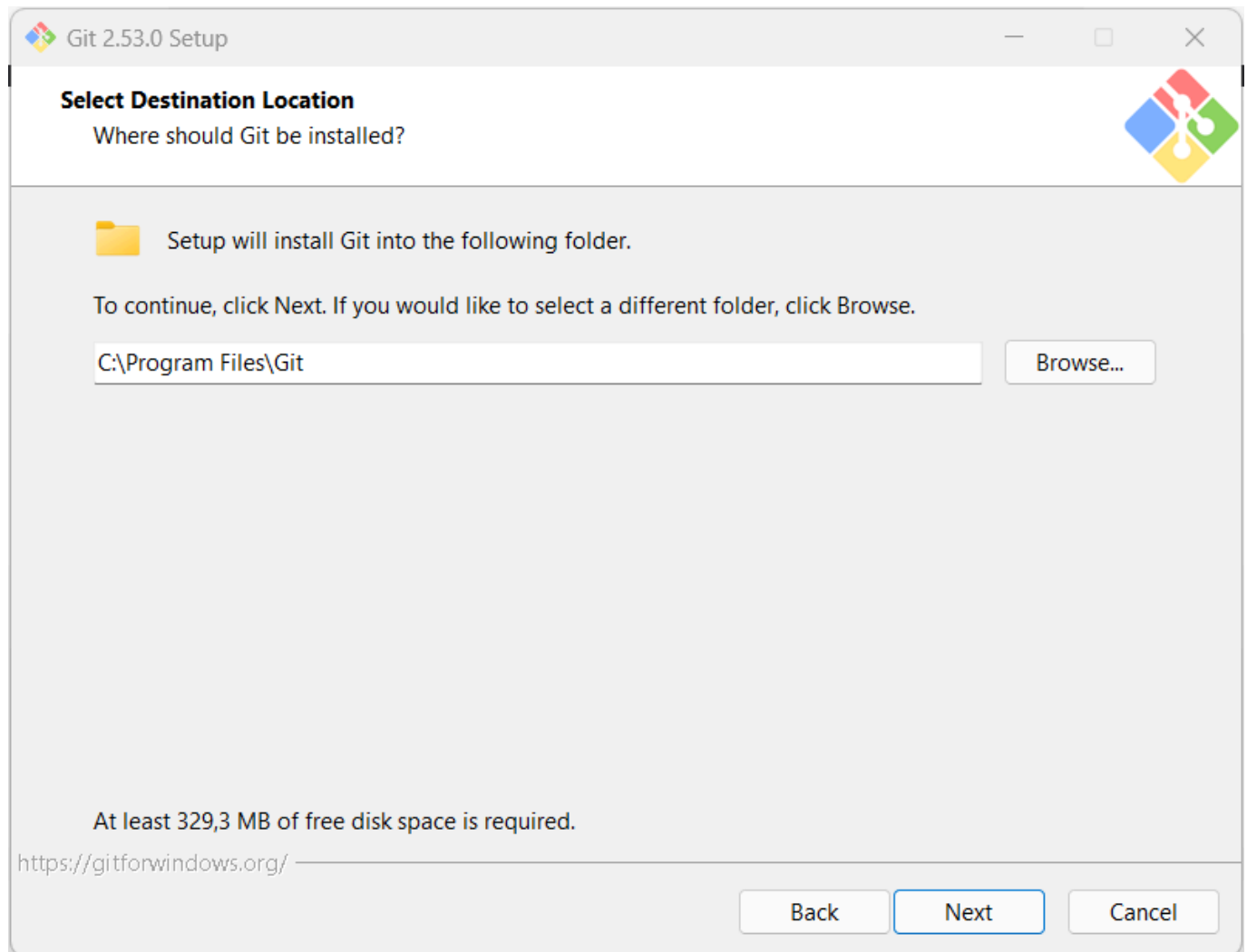
1. Download Git pada <https://git-scm.com/install/windows>



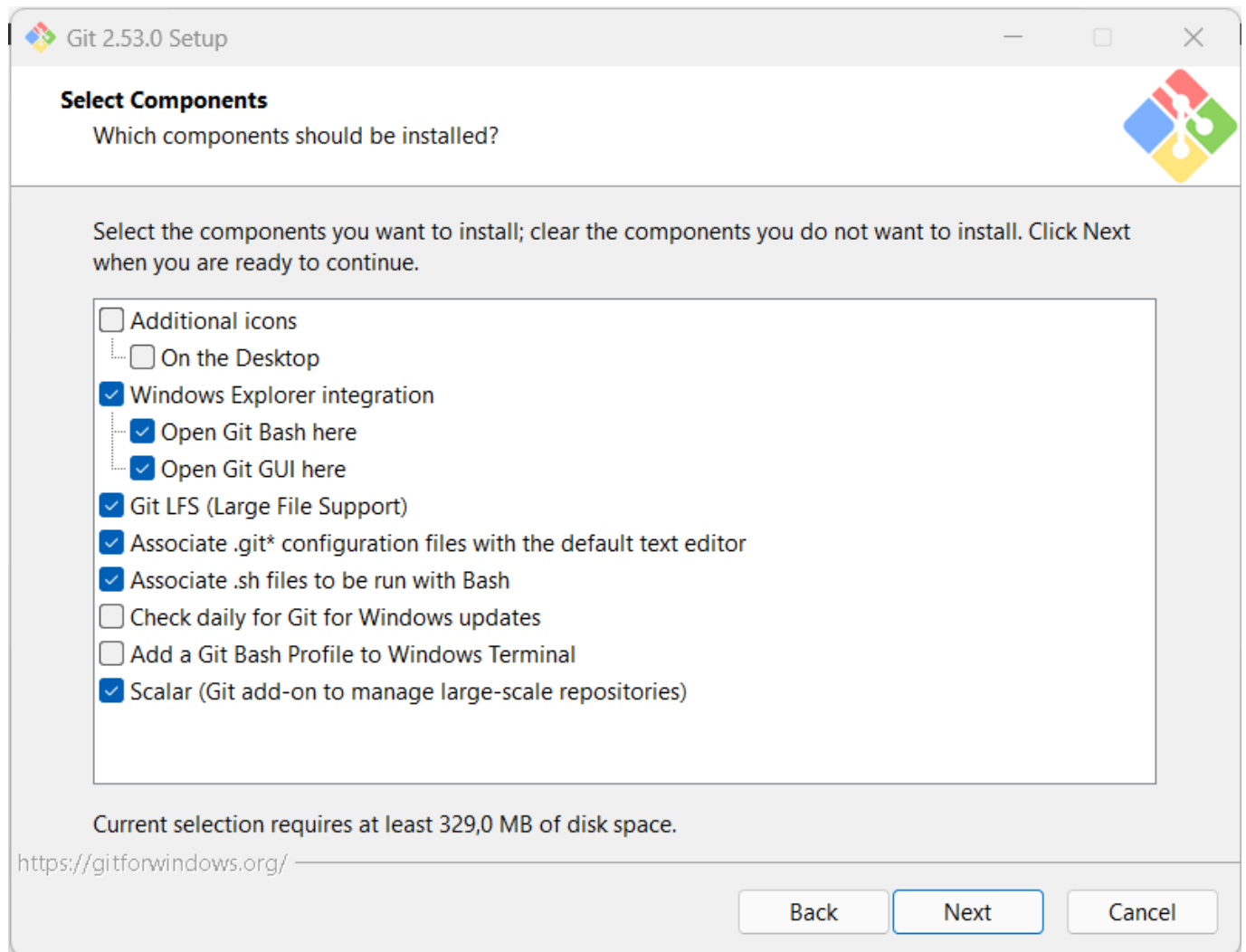
2. Klik 2x pada file .exe yang didownload, klik next



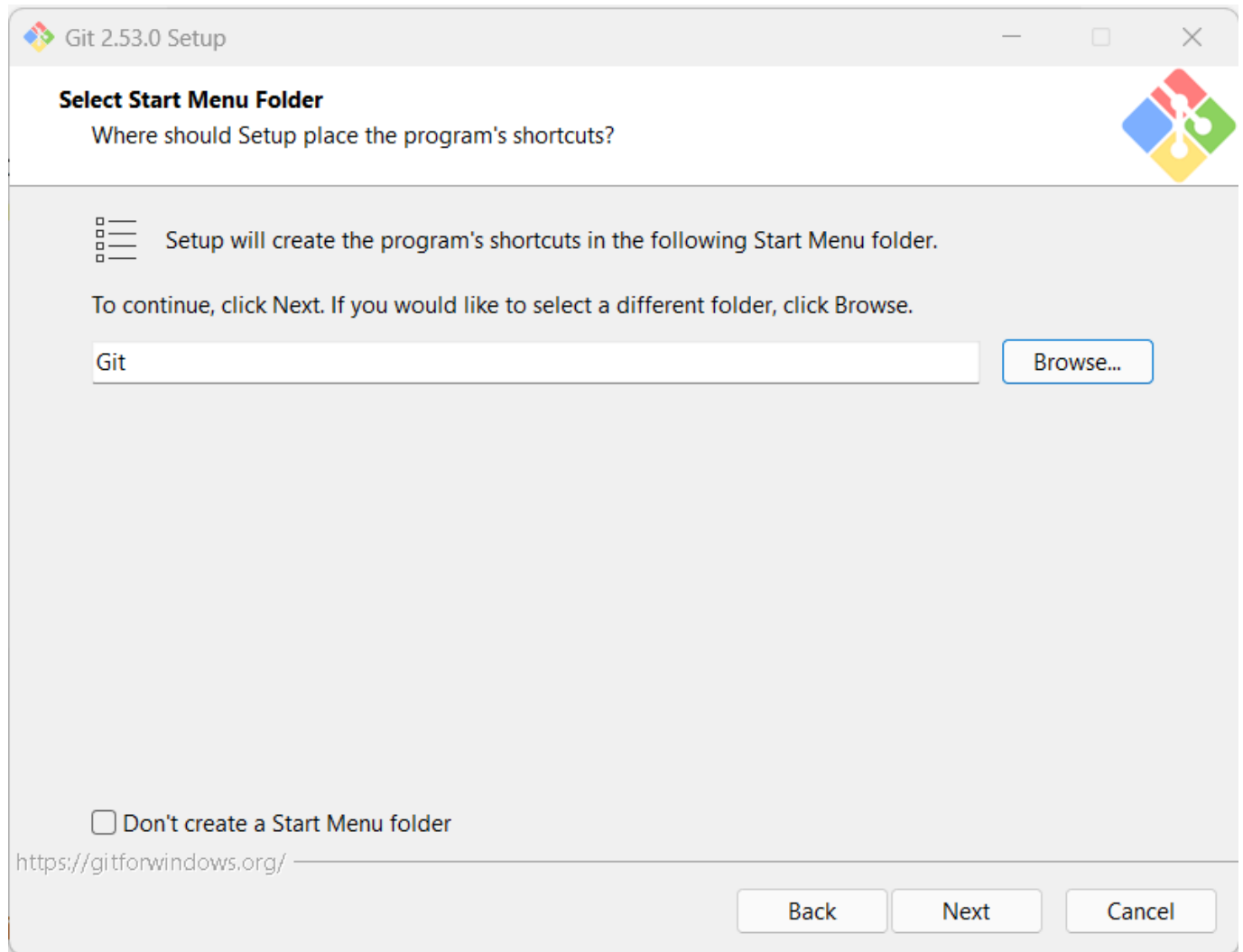
3. Pilih lokasi instalasi, klik next



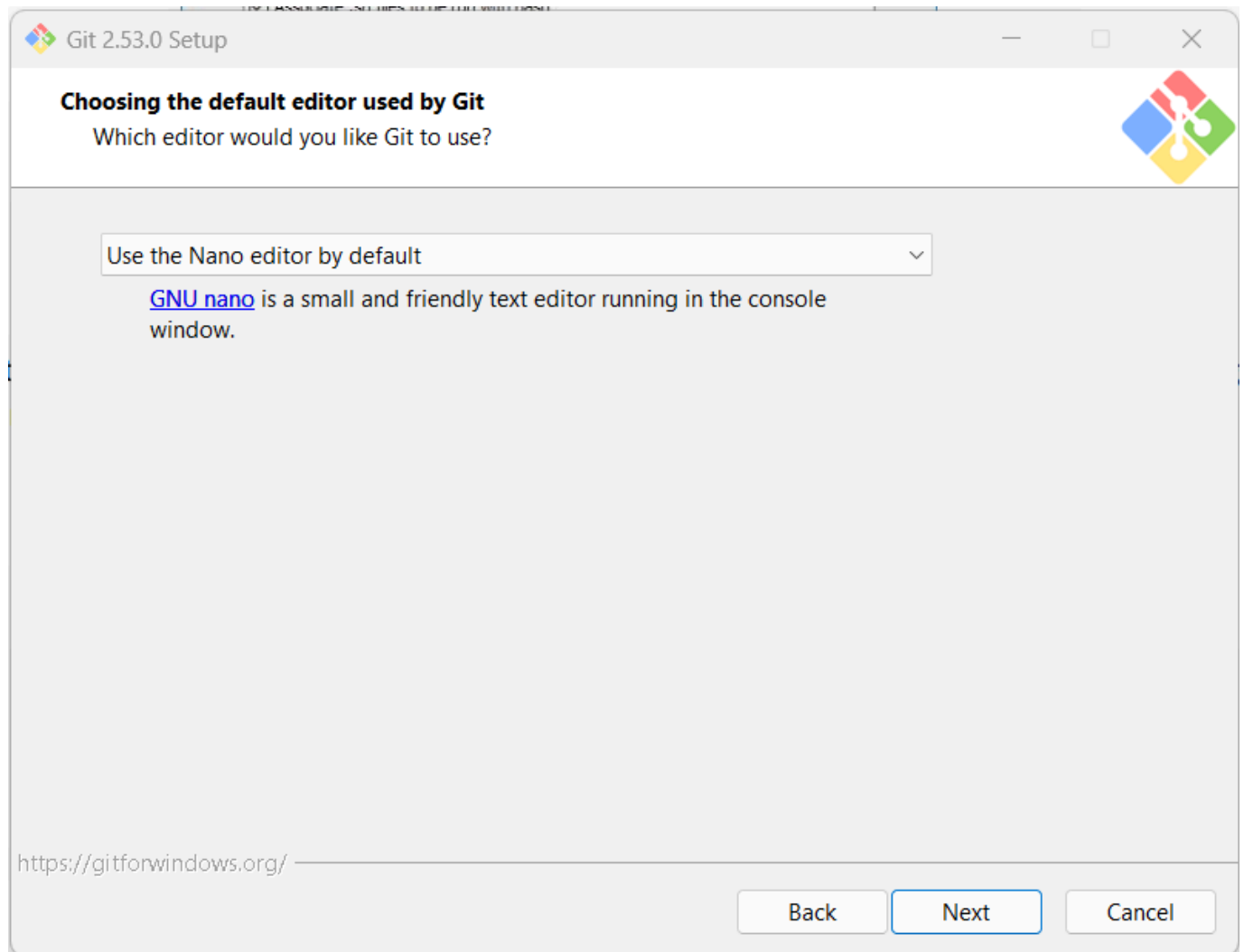
4. Pilih komponen yang ingin di install, klik next



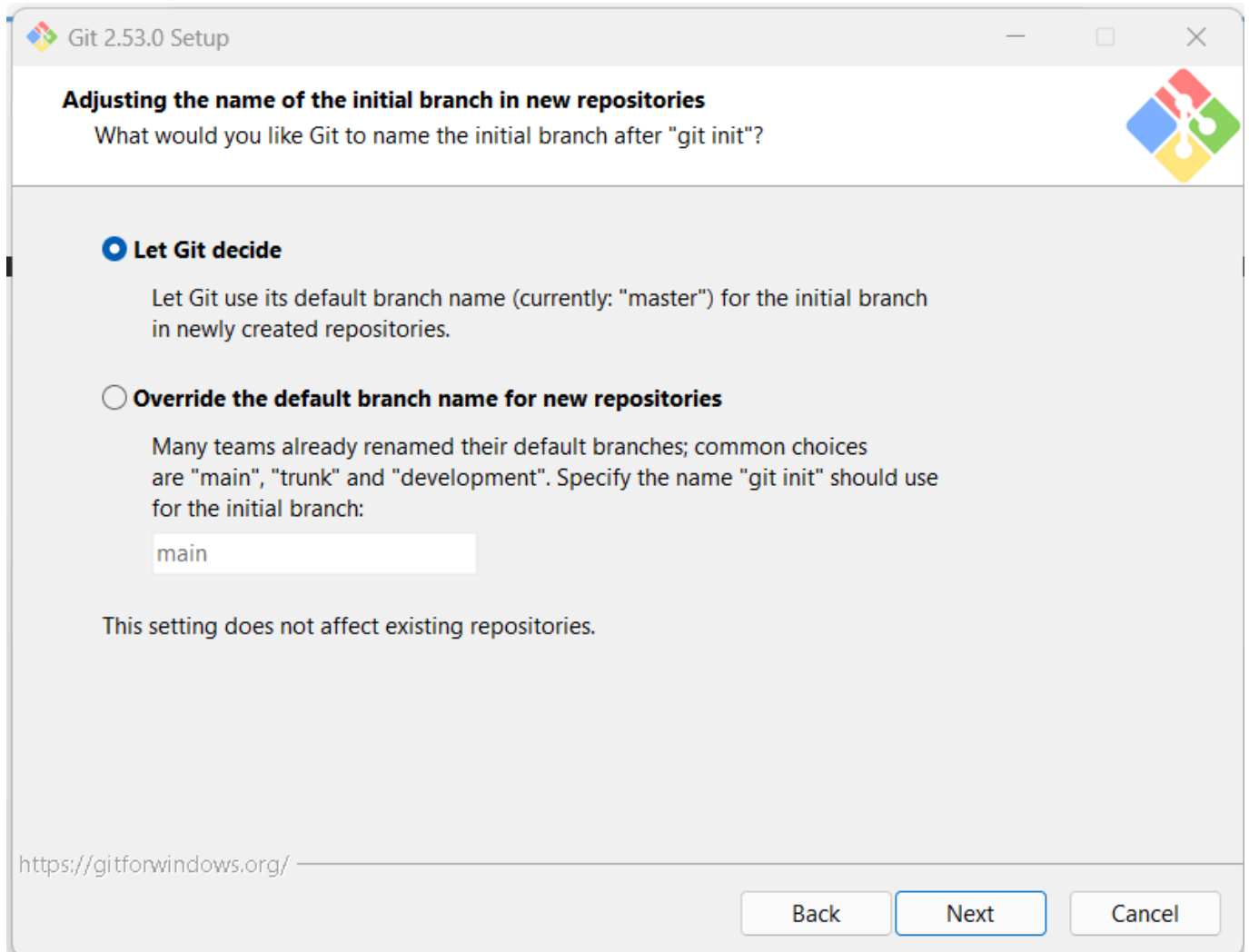
5. Select start menu folder, klik next



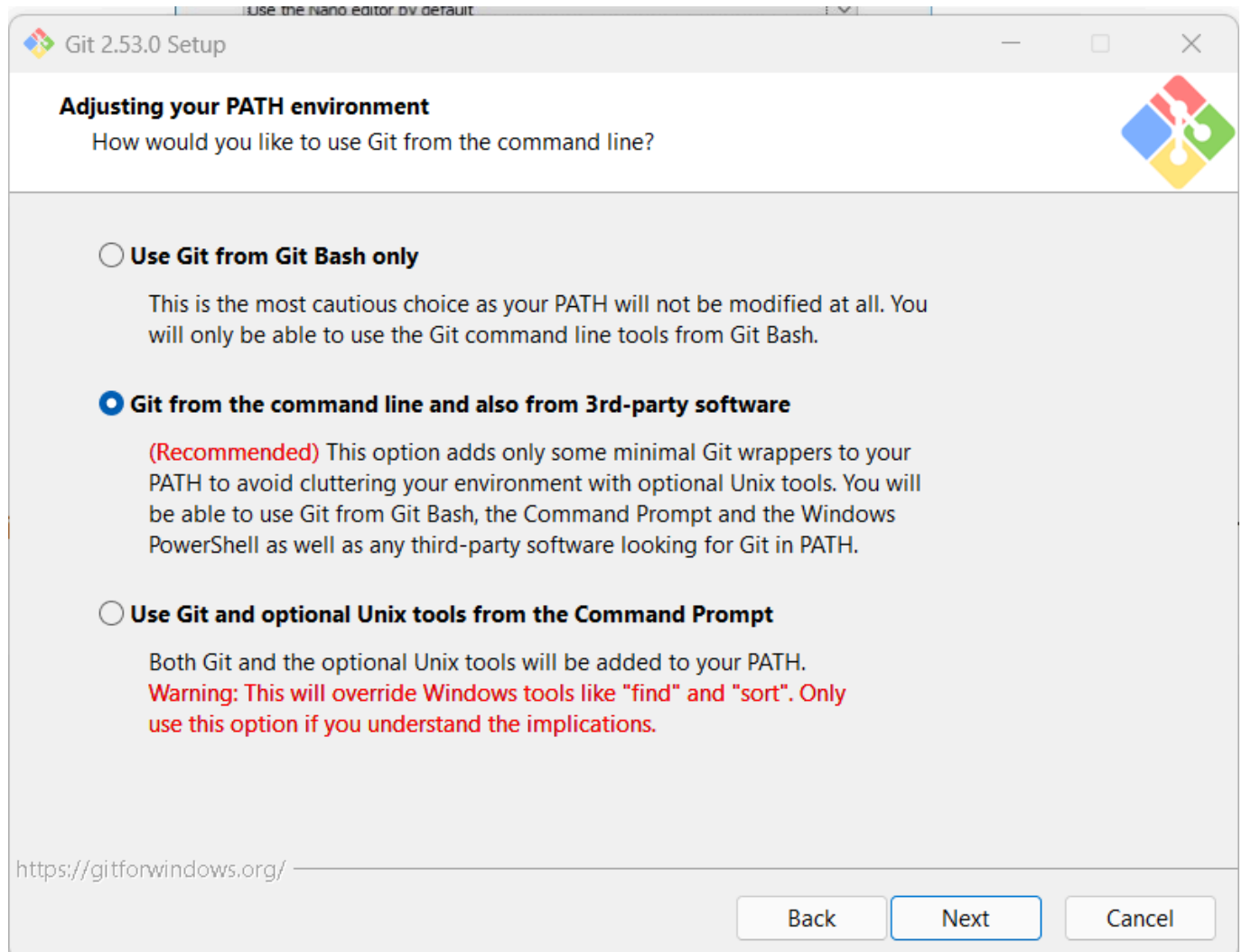
6. Pilih editor yang akan digunakan secara default oleh git (saya memilih menggunakan nano), klik next



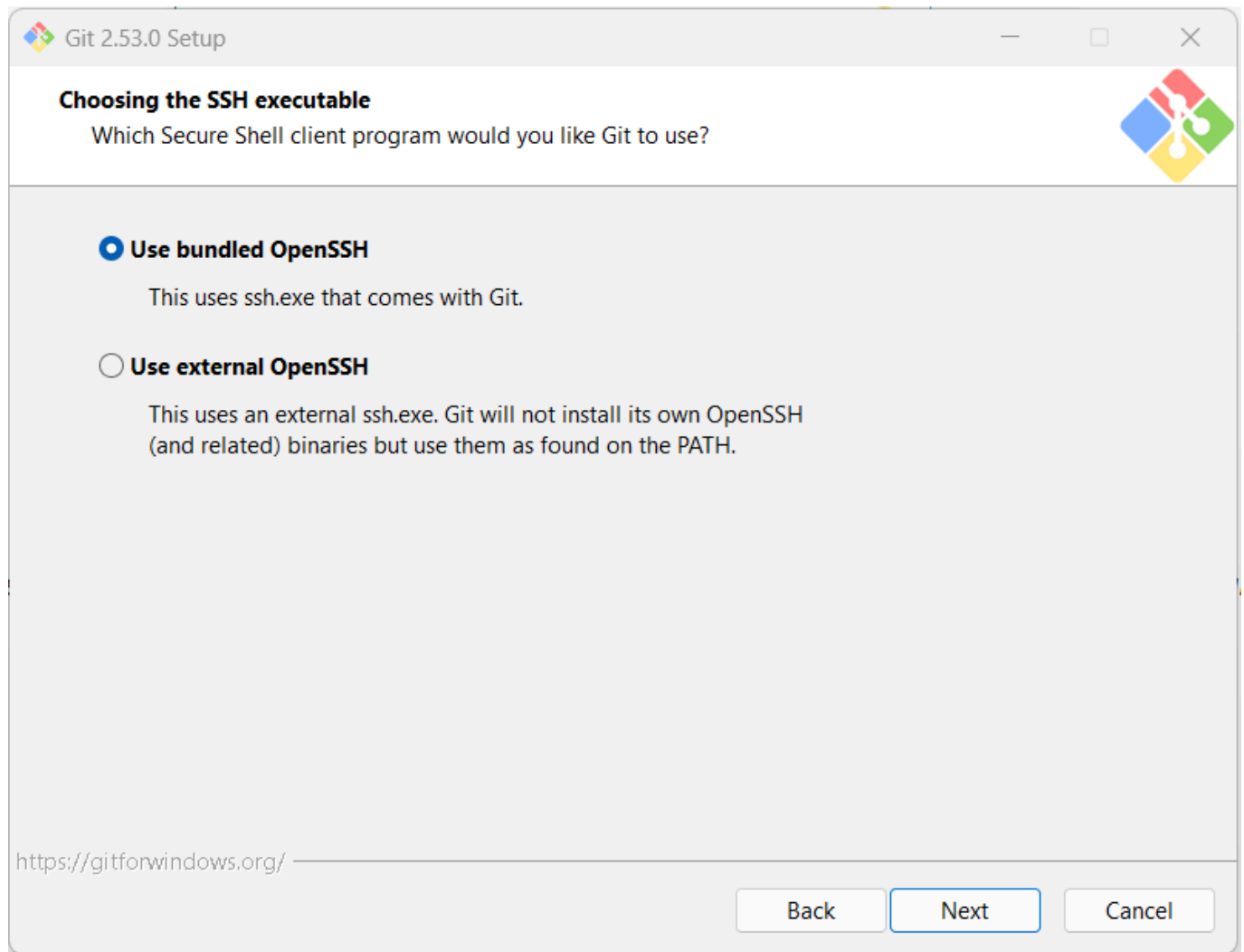
7. Pilih let git decide, klik next



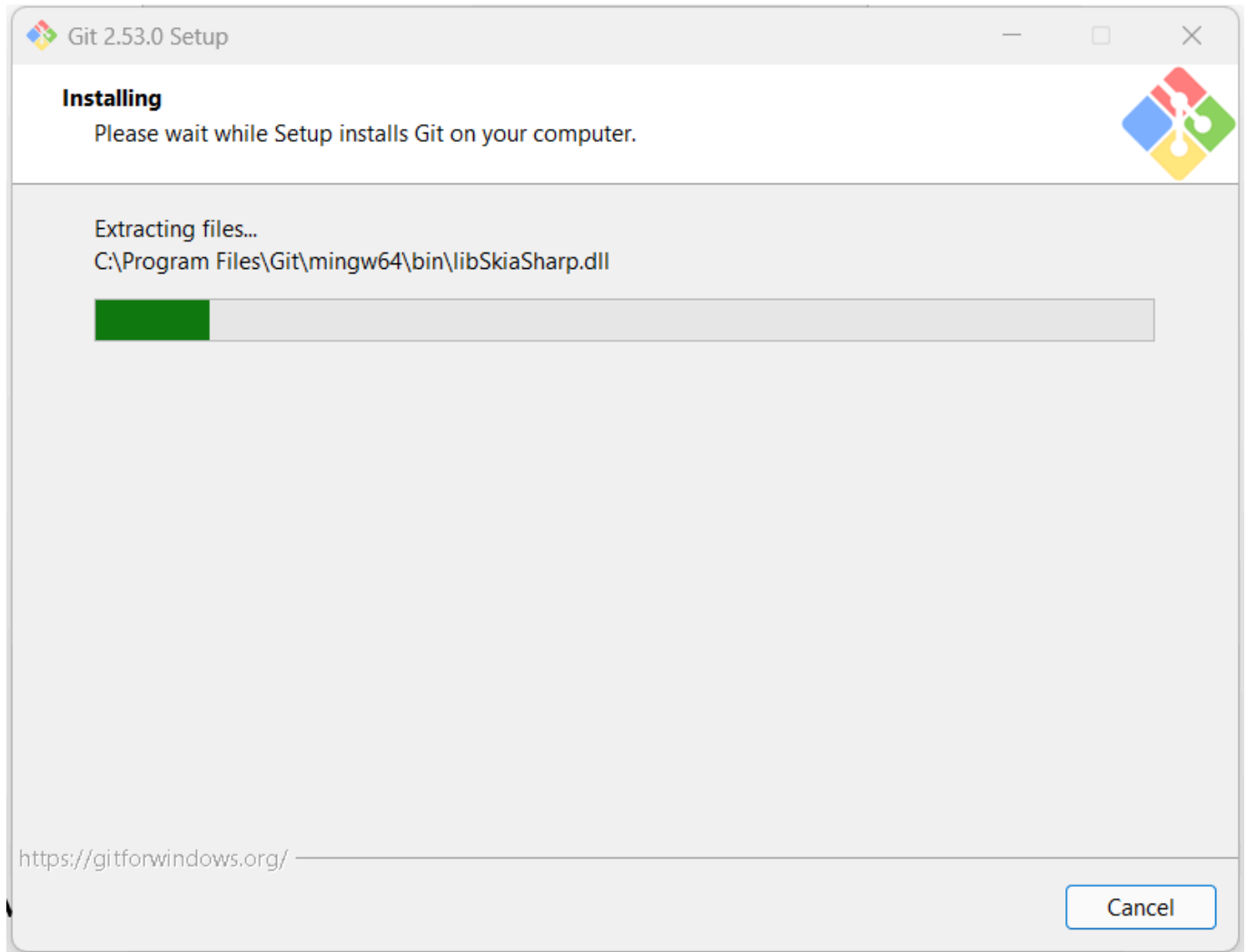
8. Pilih git from the command line and also from 3rd-party software, klik next



9. Untuk opsi selanjutnya lakukan secara default



10. Tunggu hingga proses instalasi selesai



11. Buka git bash kemudian cek versi git menggunakan command `git --version`, kemudian lakukan konfigurasi username (menggunakan command `git config --global user.name "username"`) dan email (menggunakan command `git config --global user.email email@contoh.com`). Kemudian cek konfigurasi yang dilakukan apakah sudah benar atau belum menggunakan command `git config --list`.

```
MINGW64:/c/Users/ASUS

ASUS@LAPTOP-F30FGP73 MINGW64 ~
$ git --version
git version 2.53.0.windows.1

ASUS@LAPTOP-F30FGP73 MINGW64 ~
$ git config --global user.name "Masdim37"

ASUS@LAPTOP-F30FGP73 MINGW64 ~
$ git config --global user.email dhimas.hfzh375@gmail.com

ASUS@LAPTOP-F30FGP73 MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=true
core.editor=nano.exe
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Masdim37
user.email=dhimas.hfzh375@gmail.com

ASUS@LAPTOP-F30FGP73 MINGW64 ~
$
```

Instalasi JDK

JDK (Java Development Kit) adalah paket perangkat lunak yang digunakan untuk mengembangkan aplikasi berbasis Java. JDK menyediakan berbagai tools penting, termasuk compiler untuk mengubah kode sumber Java menjadi bytecode yang dapat dijalankan oleh JVM melalui JRE.

JDK terdiri dari beberapa komponen utama:

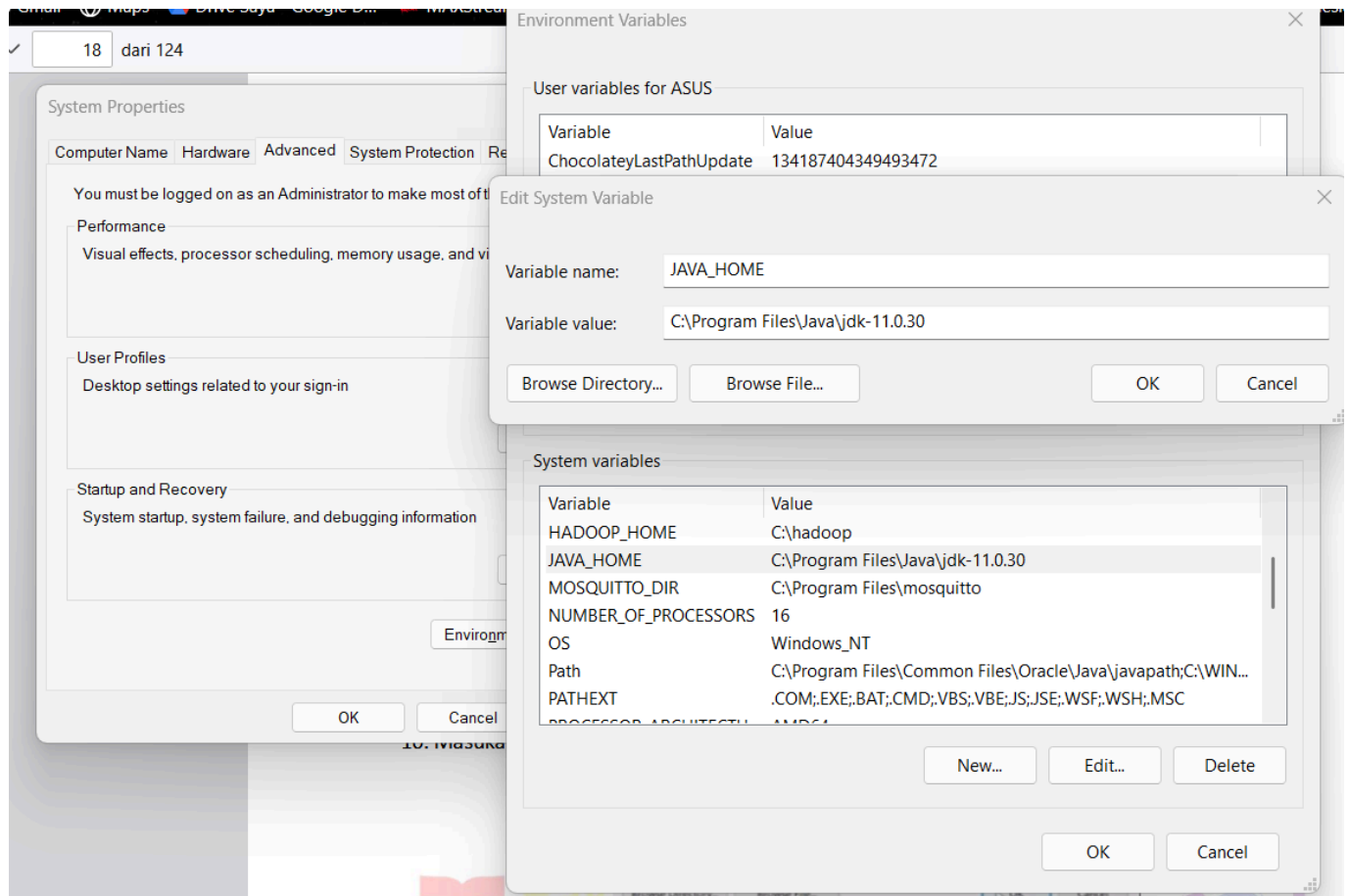
- Compiler (**javac**): mengubah kode **.java** menjadi bytecode **.class**
- JRE (Java Runtime Environment): lingkungan untuk menjalankan program Java
- JVM (Java Virtual Machine): mesin virtual yang mengeksekusi bytecode
- Tools tambahan seperti debugger dan utility lainnya

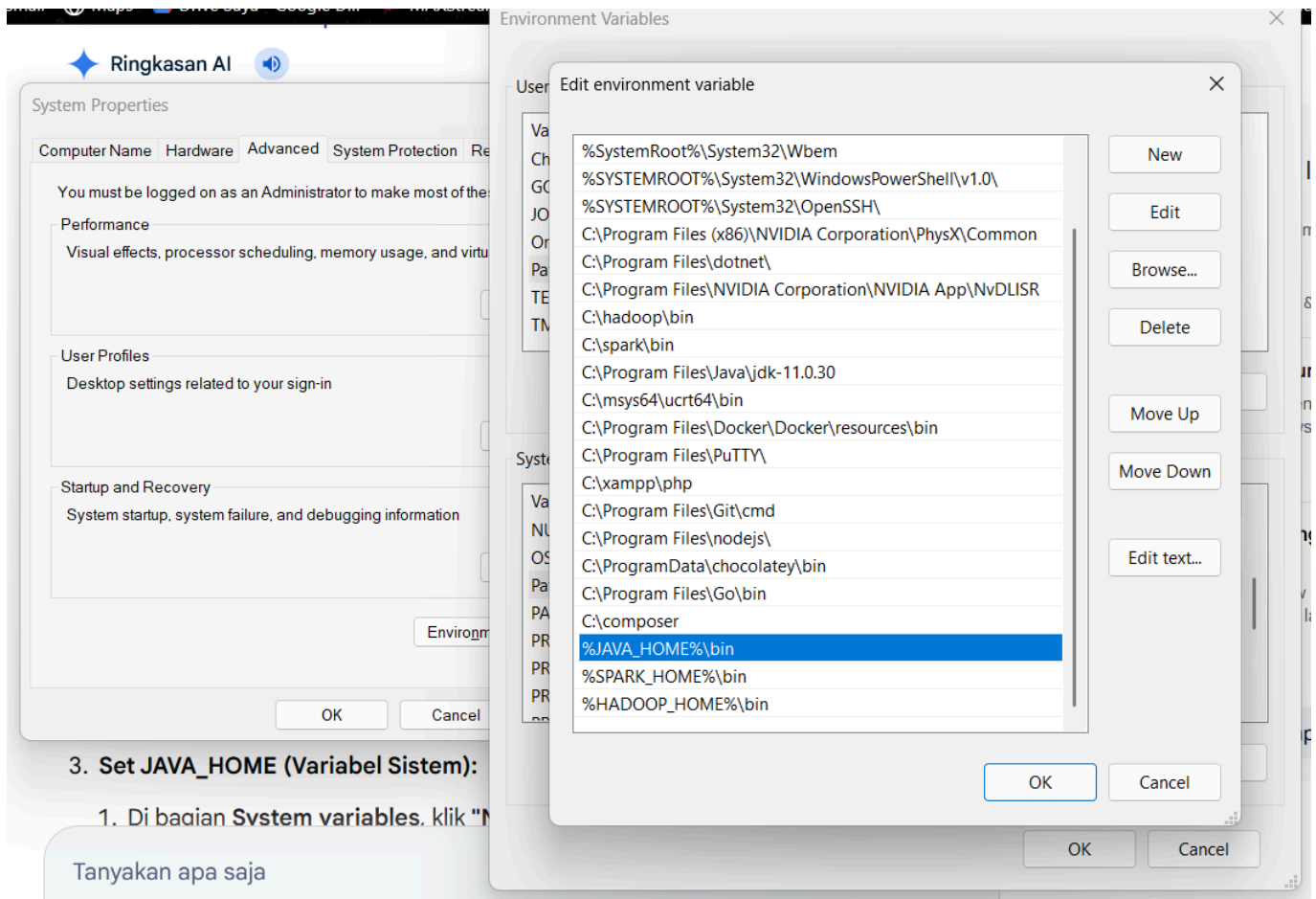
Alur kerjanya: kode Java yang ditulis developer akan dikompilasi oleh JDK menjadi bytecode, kemudian bytecode tersebut dijalankan oleh JVM yang terdapat di dalam JRE. Hal ini memungkinkan program Java bersifat platform independent (write once, run anywhere). JDK dikembangkan oleh Oracle Corporation dan menjadi komponen utama dalam proses development aplikasi Java, karena tanpa JDK, developer tidak dapat melakukan kompilasi maupun build program Java.

Berikut langkah-langkah instalasi JDK:

1. Download JDK Installer melalui <https://www.oracle.com/java/technologies/downloads/>

2. Lakukan instalasi dengan default step (karena saya sudah pernah menginstall JDK sebelumnya pada mata kuliah Pemrograman Berorientasi Objek, maka saya tidak menginstall ulang lagi)
3. Tambahkan pengaturan path environment variable JDK





3. Set JAVA_HOME (Variabel Sistem):

1. Di bagian **System variables**, klik "N

Tanyakan apa saja

4. Cek versi jdk yang sudah diinstal pada CMD

```

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ASUS>java --version
java 11.0.30 2026-01-20 LTS
Java(TM) SE Runtime Environment 18.9 (build 11.0.30+7-LTS-282)
Java HotSpot(TM) 64-Bit Server VM 18.9 (build 11.0.30+7-LTS-282, mixed mode)

C:\Users\ASUS>

```

Instalasi Flutter SDK

Flutter SDK adalah paket perangkat lunak yang digunakan untuk mengembangkan aplikasi mobile, web, dan desktop menggunakan framework Flutter. SDK ini menyediakan tools, library, dan engine yang memungkinkan developer membangun aplikasi dengan satu basis kode (single codebase). Flutter SDK dikembangkan oleh Google dan menggunakan bahasa pemrograman Dart sebagai bahasa utama. Komponen utama dalam Flutter SDK meliputi:

- Flutter Framework: kumpulan widget dan library untuk membangun UI
- Flutter Engine: mesin rendering berbasis C++ untuk menampilkan UI
- Dart SDK: compiler dan tools untuk menjalankan kode Dart
- Tools (CLI): seperti `flutter run`, `flutter build`, dan `flutter doctor`

Alur kerjanya: developer menulis kode menggunakan Dart, kemudian Flutter SDK akan mengompilasi kode tersebut (AOT/JIT) menjadi aplikasi native yang dapat dijalankan di berbagai platform.

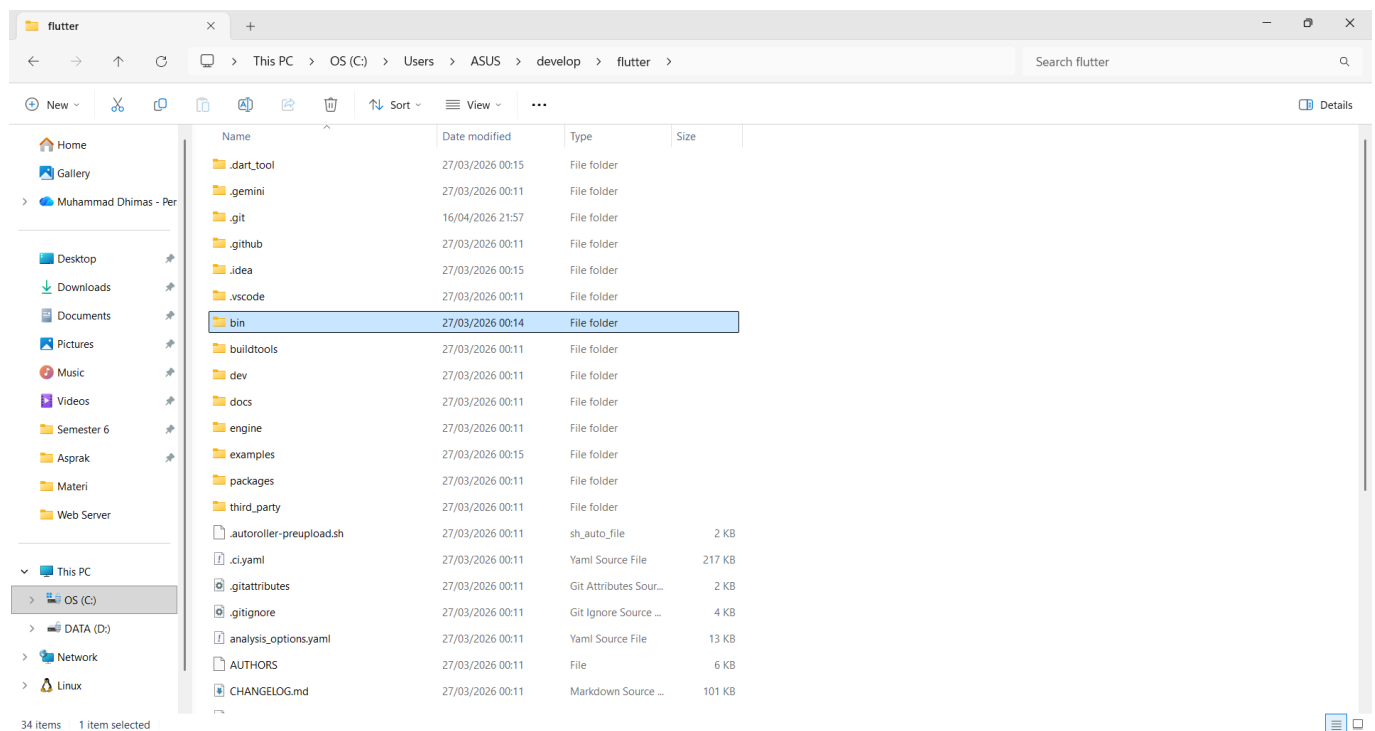
Keunggulan Flutter SDK antara lain:

- Cross-platform: satu kode untuk Android, iOS, web, dan desktop
- Hot reload: perubahan kode bisa langsung terlihat tanpa restart aplikasi
- UI fleksibel: menggunakan widget yang dapat dikustomisasi penuh

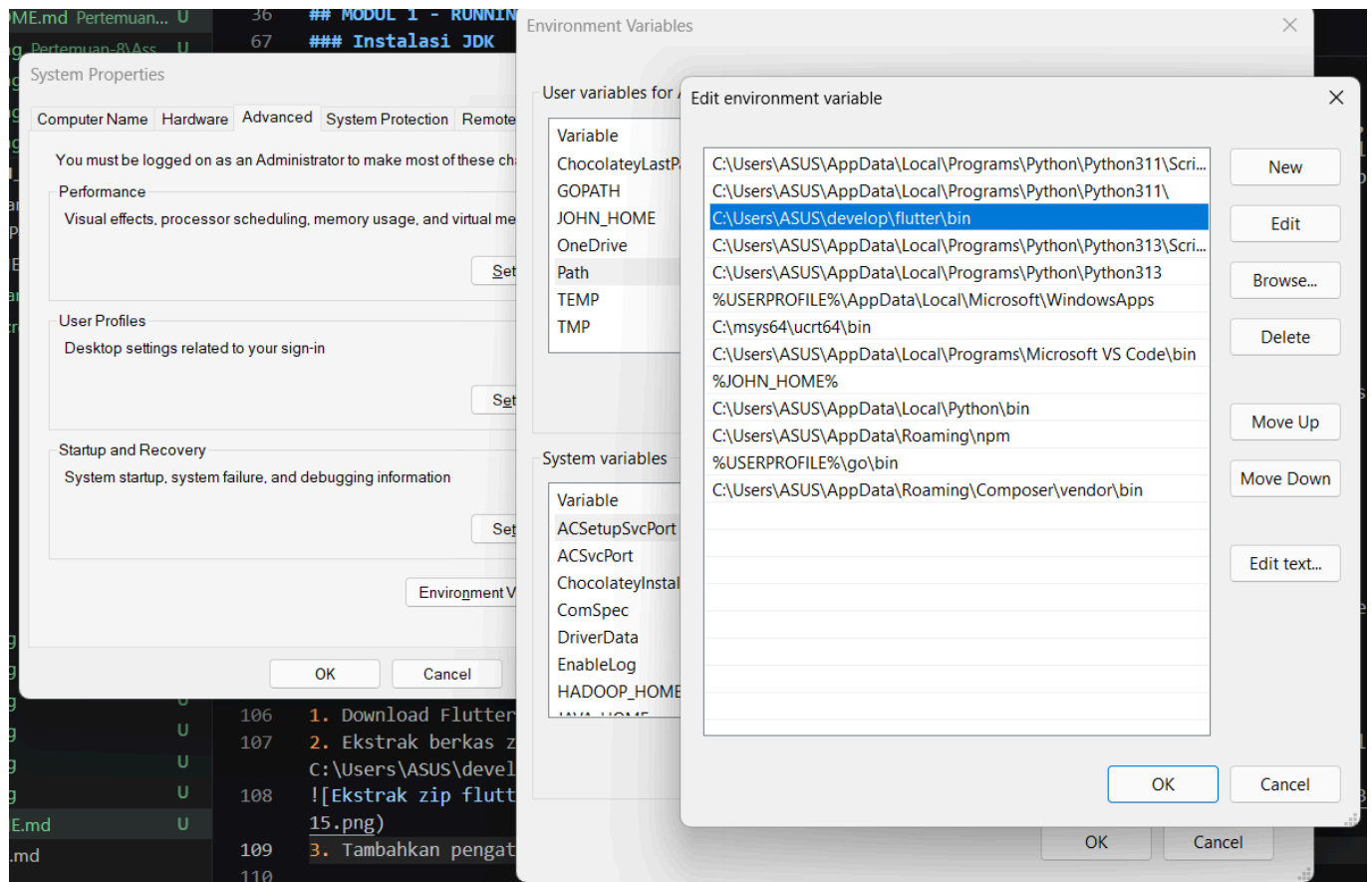
Secara keseluruhan, Flutter SDK memudahkan pengembangan aplikasi modern dengan proses yang cepat, efisien, dan konsisten di berbagai platform.

Berikut langkah-langkah instalasi Flutter SDK:

1. Download Flutter SDK secara manual pada link berikut <https://docs.flutter.dev/install/manual>
2. Ekstrak berkas zip dan tempatkan folder flutter pada lokasi instalasi yang diinginkan untuk Flutter SDK, misalnya `C:\Users\ASUS\develop\flutter`.



3. Tambahkan pengaturan path environment variable flutter



4. Jalankan Flutter Doctor. Flutter doctor merupakan perintah untuk mengecek kelengkapan framework flutter yang akan digunakan, seperti versi, Android SDK yang digunakan, iOS SDK yang digunakan (tersedia di MacOS), perangkat yang terhubung, dan lain-lain. Perintah `flutter doctor` memeriksa environment dan menampilkan laporan ke jendela terminal. Pada Flutter SDK sudah terdapat Dart SDK, jadi Anda tidak perlu menginstal Dart secara terpisah. Periksa output dengan cermat untuk perangkat lunak lain yang mungkin perlu Anda instal atau melakukan sesuatu lebih lanjut (ditunjukkan dalam teks tebal).

```

C:\Users\ASUS>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.41.6, on Microsoft Windows [Version 10.0.26200.8037], locale en-ID)
[✓] Windows Version (11 Home Single Language 64-bit, 25H2, 2009)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.1.0)
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Build Tools 2026 18.4.1)
[✓] Connected device (3 available)
[✓] Network resources

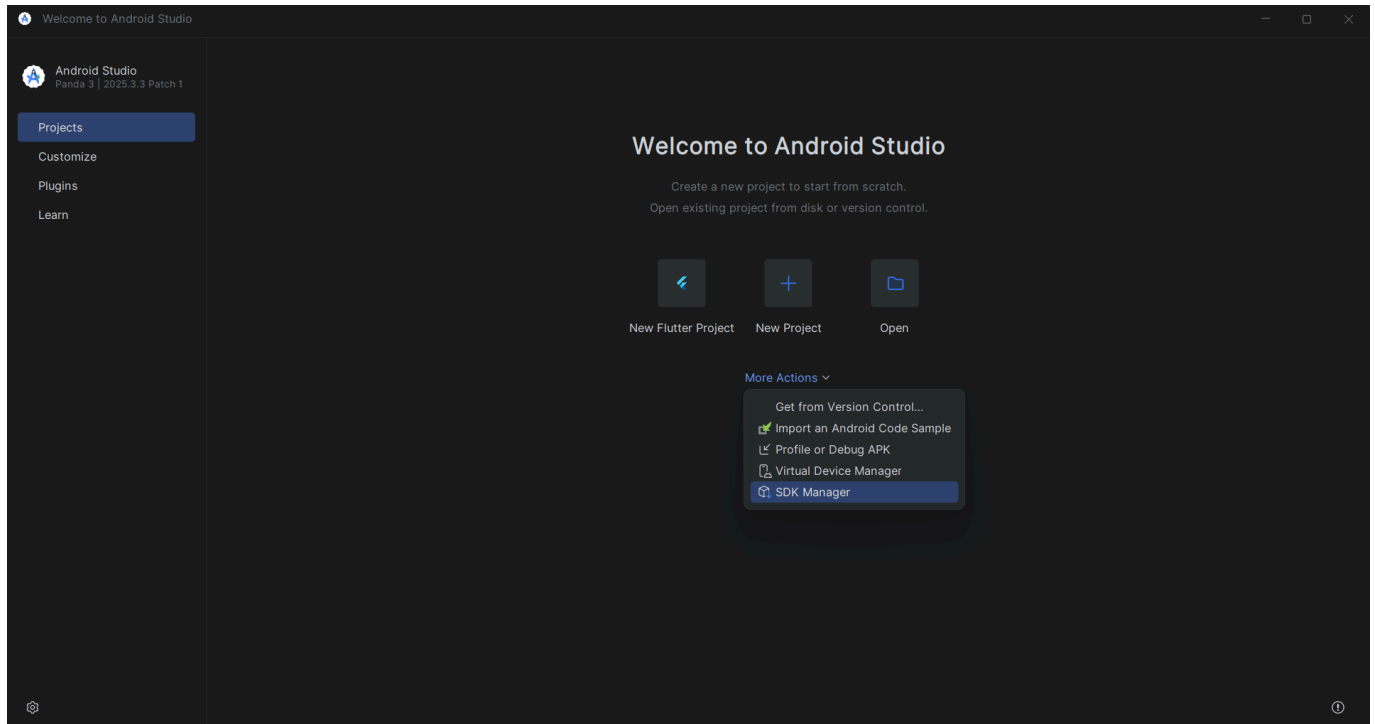
• No issues found!
C:\Users\ASUS>

```

Instalasi Android Studio

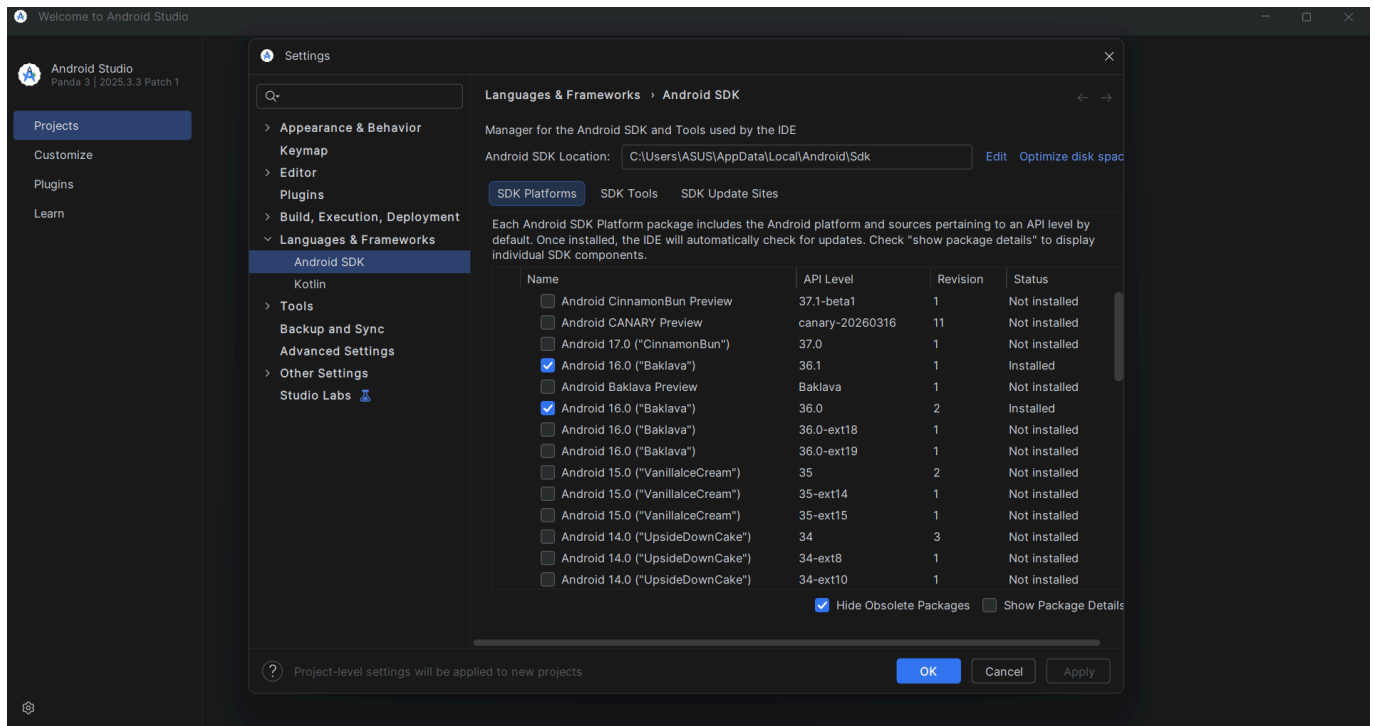
Android Studio adalah perangkat lunak (IDE) resmi yang digunakan untuk mengembangkan aplikasi Android. Dikembangkan oleh Google, Android Studio menyediakan berbagai tools terintegrasi yang memudahkan developer dalam menulis kode, mendesain antarmuka, menjalankan, hingga melakukan debugging aplikasi. Di dalamnya terdapat komponen penting seperti code editor, Android SDK, emulator, serta build tools berbasis Gradle yang digunakan untuk proses build dan manajemen dependensi. Alur kerjanya dimulai dari penulisan

kode (menggunakan Java atau Kotlin), kemudian aplikasi di-build dan dijalankan melalui emulator atau perangkat Android secara langsung. Dengan fitur yang lengkap dan terintegrasi, Android Studio menjadi solusi utama dalam pengembangan aplikasi Android karena mampu meningkatkan efisiensi dan produktivitas developer dalam satu lingkungan kerja.



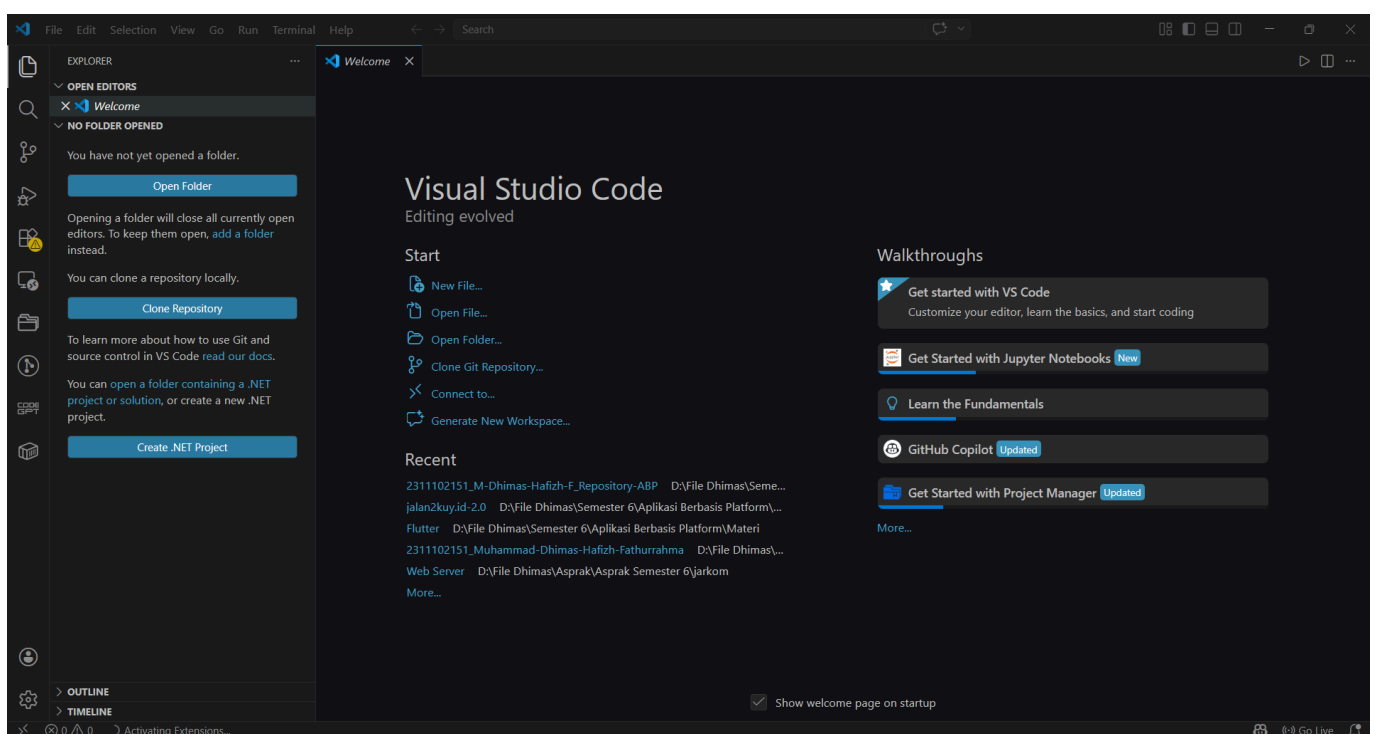
Instalasi SDK Android

Android SDK (Software Development Kit) adalah kumpulan alat, library, dan komponen yang digunakan untuk mengembangkan aplikasi berbasis Android. SDK ini menyediakan berbagai kebutuhan penting seperti API, emulator, serta tools build yang memungkinkan developer membuat, menguji, dan menjalankan aplikasi Android secara efisien. Android SDK merupakan bagian utama dalam proses development yang biasanya terintegrasi dengan Android Studio dan dikembangkan oleh Google. Di dalamnya terdapat komponen seperti platform Android (API level), build tools, serta debugging tools yang membantu dalam proses pengembangan hingga deployment aplikasi. Dengan adanya Android SDK, developer dapat memastikan aplikasi yang dibuat kompatibel dengan berbagai versi sistem operasi Android serta perangkat yang berbeda-beda. Android SDK dapat didownload di Android Studio pada menu **Settings - Languages & Framework - Android SDK**.



Instalasi Visual Studio Code

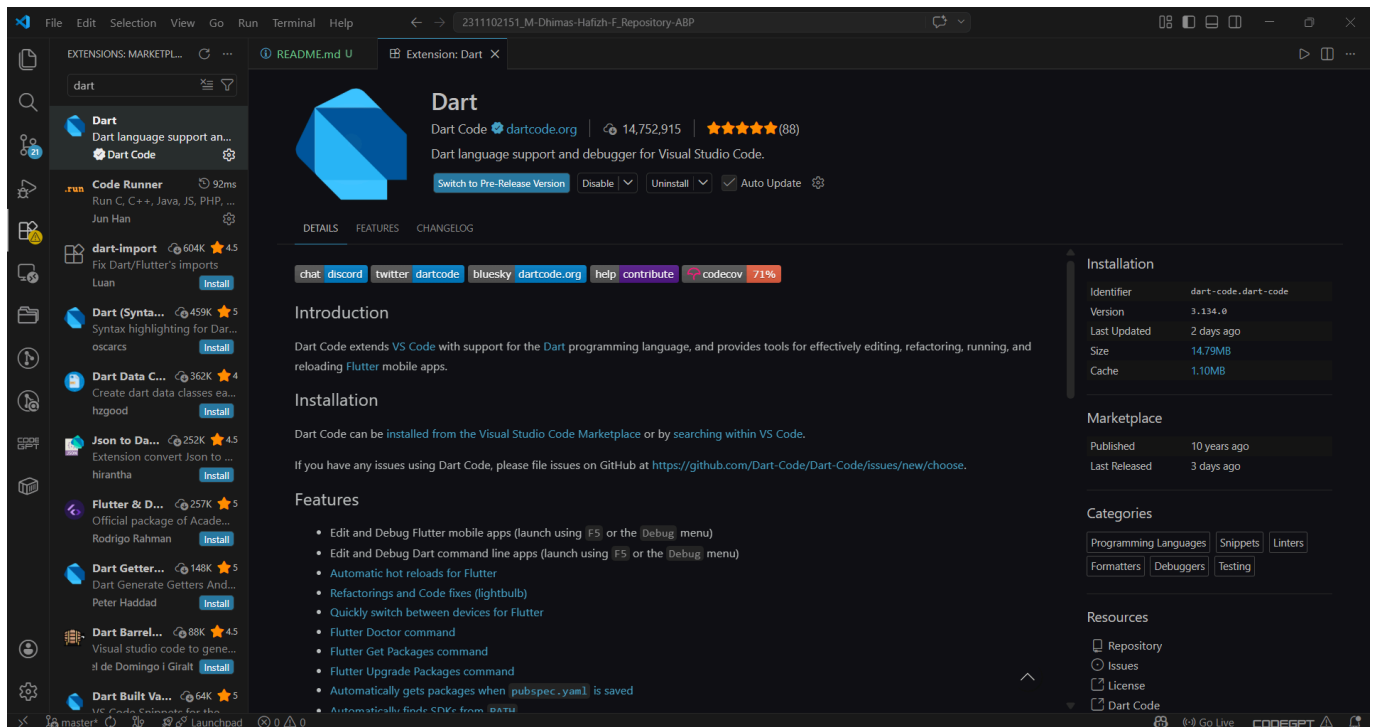
Visual Studio Code adalah perangkat lunak editor kode sumber (code editor) yang ringan namun kuat untuk pengembangan berbagai jenis aplikasi. Dikembangkan oleh Microsoft, Visual Studio Code mendukung banyak bahasa pemrograman seperti Java, Python, JavaScript, dan Dart, serta dapat digunakan untuk pengembangan web, mobile, hingga backend. Editor ini dilengkapi dengan fitur penting seperti IntelliSense (auto-completion), debugging, terminal terintegrasi, serta dukungan ekstensi yang sangat luas untuk menambah fungsionalitas sesuai kebutuhan. Dengan bantuan ekstensi, Visual Studio Code dapat diintegrasikan dengan berbagai framework dan tools seperti Flutter, Git, dan Docker. Karena sifatnya yang ringan, fleksibel, dan mudah dikustomisasi, Visual Studio Code menjadi salah satu editor paling populer yang digunakan oleh developer untuk meningkatkan produktivitas dalam proses coding.



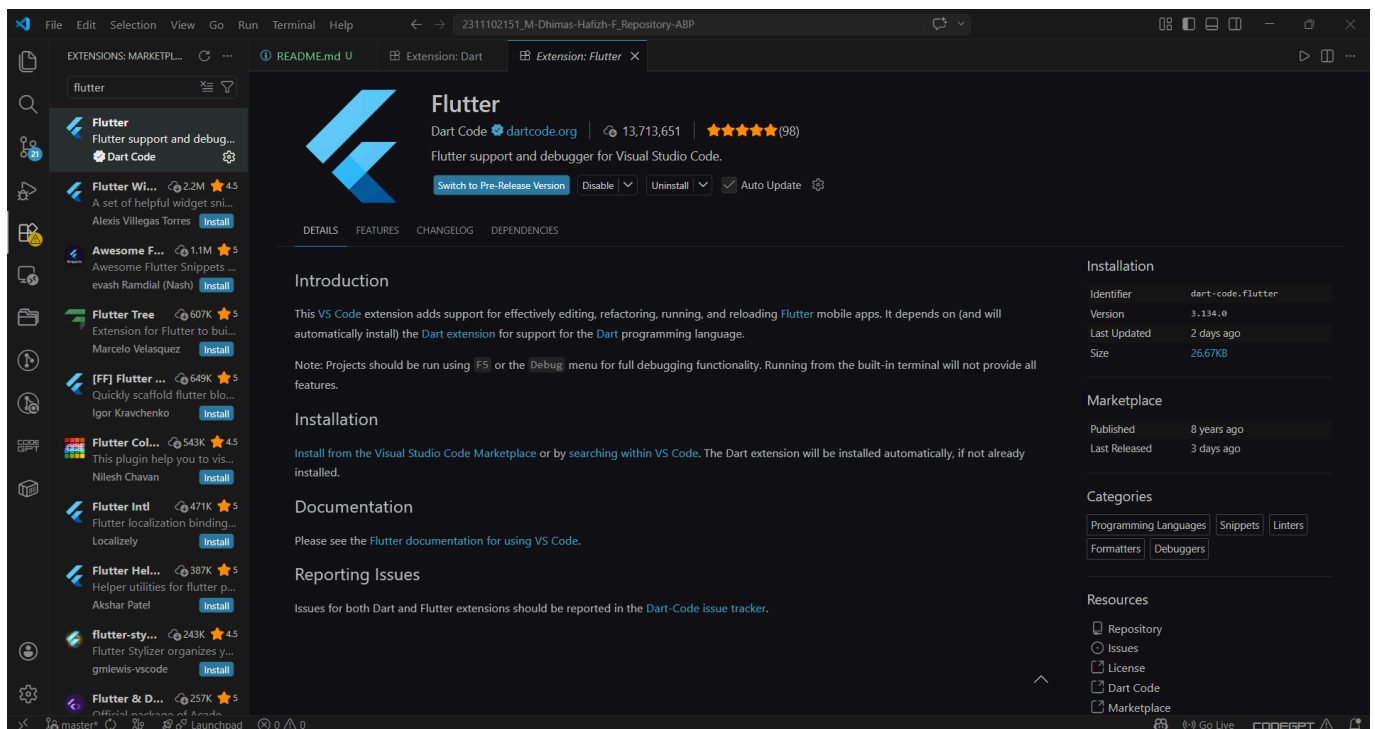
Instalasi Extension Visual Studio Code

Beberapa extension sangat diperlukan untuk diinstall yang nantinya dipakai sebagai pendukung ketika membuat aplikasi menggunakan Flutter. Berikut extension yang perlu diinstal:

1. Dart



2. Flutter

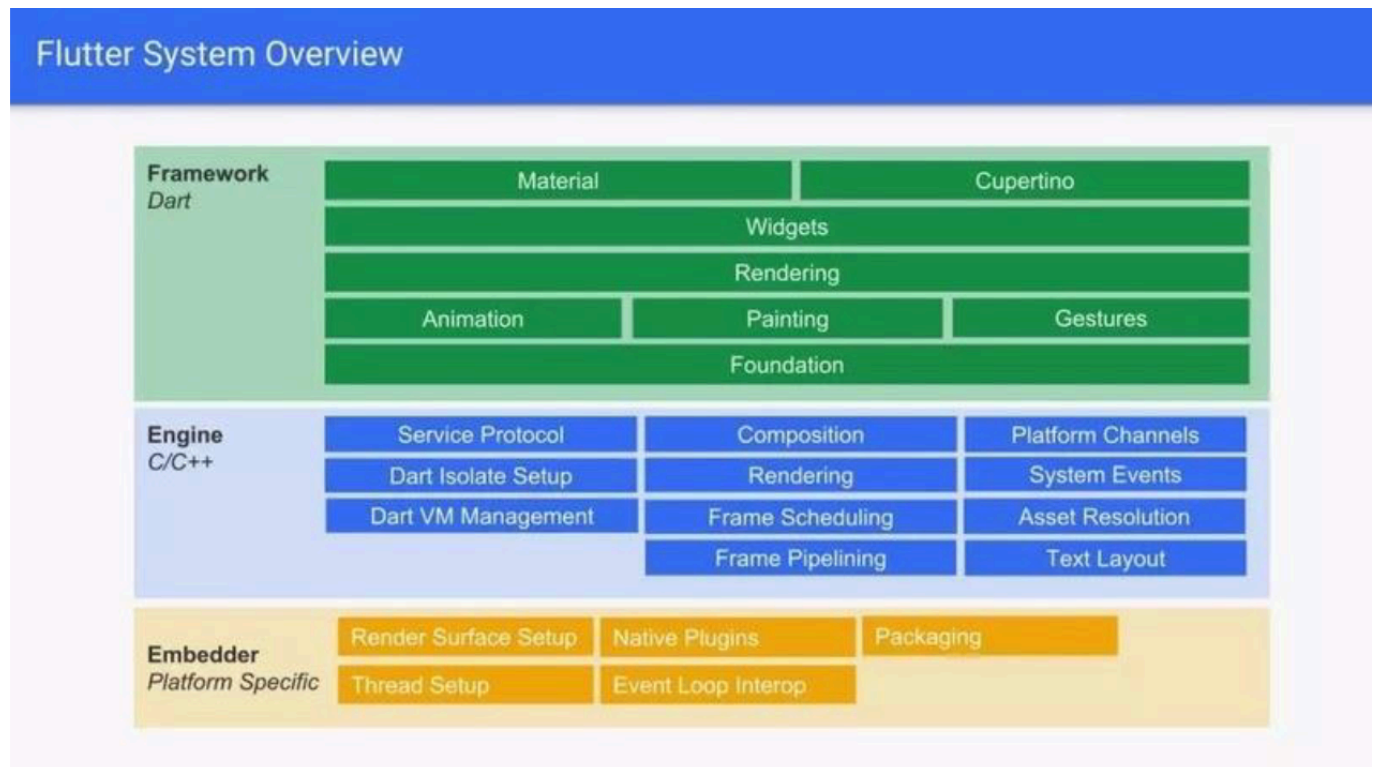


MODUL 2 - PENGENALAN FLUTTER

Apa Itu Flutter

Flutter adalah framework UI yang dikembangkan oleh Google untuk membangun aplikasi cross-platform. Flutter menggunakan bahasa Dart, sementara engine-nya ditulis dengan C dan C++ serta memanfaatkan Skia Graphics Engine untuk merender tampilan secara langsung ke layar. Engine ini juga digunakan pada berbagai produk besar seperti Google Chrome dan Mozilla Firefox, sehingga performanya sudah teruji.

Flutter berjalan menggunakan Dart Virtual Machine (VM) yang mendukung kompilasi just-in-time (JIT) saat development, sehingga memungkinkan fitur hot reload untuk melihat perubahan kode secara instan tanpa restart aplikasi. Saat tahap produksi, Flutter menggunakan kompilasi ahead-of-time (AOT) agar aplikasi memiliki performa yang optimal.



Dalam pengembangan UI, Flutter menggunakan konsep widget tree, yaitu struktur hierarki widget di mana setiap komponen antarmuka dibangun dari widget yang dapat bersarang satu sama lain. Widget dibagi menjadi dua jenis utama: stateless widget (tidak memiliki state tetap) dan stateful widget (memiliki state yang dapat berubah selama aplikasi berjalan). Konsep ini membantu developer dalam mengelola tampilan dan logika aplikasi secara terstruktur, modular, dan efisien.

Arsitektur Flutter

Selain konsep dasar, Flutter juga memiliki pendekatan arsitektur untuk mengelola alur data dan state aplikasi secara lebih terstruktur. Salah satu arsitektur yang umum digunakan adalah BLoC (Business Logic Component). Pendekatan ini berfokus pada bagaimana aplikasi merespons event (aksi dari user atau sistem) dan mengubah state (kondisi aplikasi) secara terkontrol.

Dalam arsitektur BLoC, logika bisnis dipisahkan dari tampilan (UI), sehingga UI hanya bertugas menampilkan data, sementara proses seperti pengolahan data, validasi, dan pengambilan keputusan ditangani oleh BLoC. Alur kerjanya umumnya: event → diproses oleh BLoC → menghasilkan state baru → UI diperbarui.

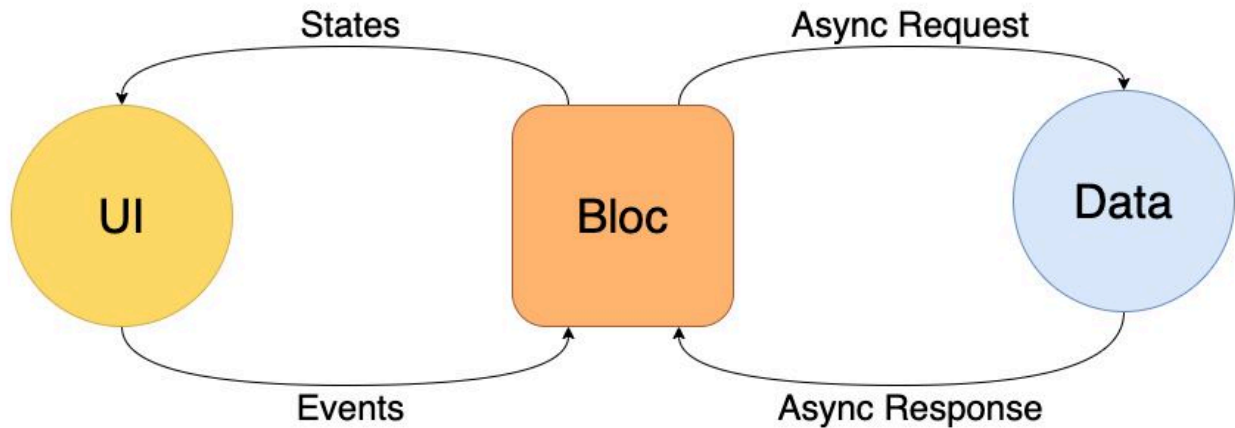
Keunggulan utama BLoC terletak pada:

- **Simplicity:** alur data jelas dan terstruktur
- **Scalability:** mudah dikembangkan untuk aplikasi besar

- Testability: logika bisnis mudah diuji secara terpisah dari UI

Dengan arsitektur ini, pengembangan aplikasi Flutter menjadi lebih rapi, modular, dan mudah dikelola, terutama ketika aplikasi semakin kompleks.

Architecture



Hello World Pada Flutter

Pada pengenalan Flutter kali ini, kita akan membuat Hello World sebagai permulaan ketika menggunakan Flutter. Contoh kode hello world pada flutter yang ditulis dengan nama kode `main.dart`

```
// ignore_for_file: prefer_const_constructors

import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  // Root aplikasi
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Hello World",
      home: const MyHomePage(
        title: "Flutter Hello World Page",
      ),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({
    Key? key,
```

```
        required this.title,
    }) : super(key: key);

    final String title;

    @override
    State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(
                title: Text(widget.title),
            ),
            body: const Center(
                child: Text(
                    'Hello World',
                ),
            ),
        );
    }
}
```

Output :

00.24 280
B/SVo
LTE

4G



4G+



82%

DEBUG

Flutter Hello World Page

Hello World



MODUL 3 - PENGENALAN DART

Pengenalan Dart

Dart adalah bahasa pemrograman yang digunakan dalam pengembangan aplikasi Flutter, dikembangkan oleh Google. Untuk mulai belajar Flutter, tidak harus menguasai Dart secara mendalam, namun penting memahami konsep dasar seperti variabel, percabangan (control statement), perulangan (looping), array/list, dan fungsi. Secara sintaks, Dart memiliki kemiripan dengan bahasa seperti C dan Java, misalnya penggunaan kurung kurawal {}, struktur program, serta kewajiban penggunaan tanda titik koma ; di akhir pernyataan. Dart juga mendukung pemrograman berorientasi objek (OOP), sehingga memungkinkan pembuatan kode yang terstruktur dan modular.

Dengan memahami dasar-dasar tersebut, developer sudah dapat mulai membangun aplikasi Flutter secara efektif, sambil memperdalam Dart seiring proses pengembangan.

Variable

Untuk penggunaan variable di dart, terdapat beberapa cara, yaitu dengan var, type annotation dan multiple variable.

```
//var
var variableName;
var name = "Dhimas";

//type annotation
String? nama;
```



```
String namaLengkap = "M Dhimas Hafizh F";
int? a, b, c;

//multiple variable
int x = 1, y = 2, z = 3;
```

Variable primitive yang tersedia di dart : integer, double, string, boolean

Statement Control

Terdapat beberapa cara untuk mendeklarasikan statement control, yaitu if, if else, if else if, switch case.

IF Statement

```
void main() {
    int angka = 6;

    //IF Statement
    if (angka % 2 == 0) { //condition
        print("Angka Genap"); //statements
    }
}
```

IF ELSE Statement

```
void main() {
    int angka = 7;

    //IF Statement
    if (angka % 2 == 0) { //condition
        print("Angka Genap"); //statements
    } else {
        print("Angka Ganjil"); //statements
    }
}
```

IF ELSE IF Statement

```
void main() {
    int nilai = 73;
    if (nilai >= 85) { //condition 1
        print("Grade A"); //statements 1
    } else if (nilai >= 75) { //condition 2
        print("Grade B"); //statements 2
    } else if (nilai >= 65) { //condition 3
        print("Grade C"); //statements 3
    }
}
```

```
    } else { //else
        print("Grade D"); //statements 4
    }
}
```

Switch Case

```
void main() {
    String hewan = "Ayam";

    switch (hewan) { //switch(expression)
        case "Kucing": //case value 1
            print("berkaki 4"); //statements
            break;
        case "Ayam": //case value 2
            print("berkaki 2"); //statements
            break;
        default:
            print("tidak valid"); //statements
            break;
    }
}
```

Looping

Secara umum, terdapat dua cara untuk melakukan looping di dart, yaitu menggunakan for loop dan while loop.

FOR loops

Gunakan for loop saat kondisinya tau persis seberapa banyak looping akan dilakukan, contohnya melakukan perulangan sebanyak 10 kali dengan iterasi sebanyak 1 tingkat atau 1 kali.

```
void main() {
    for (int angka = 0; angka < 10; angka++) {
        print("Angka = " + angka.toString());
    }
}
```

WHILE loops

Gunakan while loop saat kondisinya tidak tahu kapan perulangan akan berhenti, contohnya sediakan input angka hingga user menginput tanda "-".

```
void main() {  
    int angka = 0;  
    while (angka < 5) { //expression  
        print("Angka = " + angka.toString()); // Statement(s) to be executed if  
        expression is true  
        angka++;  
    }  
}
```

List

Secara umum, kumpulan banyak data dalam satu variable disebut array. Tetapi beberapa bahasa pemrograman menyebutnya dengan list, termasuk bahasa dart ini. List memiliki 2 tipe, yaitu Fixed Length List dan Growable List.

Fixed Length List

Tipe list ini memiliki panjang index yang tetap dan tidak dapat bertambah banyak.

```
void main() {  
    var listAngka = List.filled(3, 0);  
  
    listAngka[0] = 15;  
    listAngka[1] = 44;  
    listAngka[2] = 98;  
  
    print(listAngka);  
  
    var listNama = List.filled(3, "");  
  
    listNama[0] = "Dhimas";  
    listNama[1] = "Hafizh";  
    listNama[2] = "Fathur";  
  
    print(listNama);  
}
```

Growable List

Gunakan growable list apabila memiliki banyak object yang tidak menentu atau banyaknya object yang terus bertambah.

```
void main() {  
    List<int> listAngka = [];  
  
    listAngka.add(10);  
    listAngka.add(20);  
}
```

```
listAngka.add(30);
listAngka.add(40);

print(listAngka);

List<String> listNama = [];

listNama.add("Dhimas");
listNama.add("Hafizh");
listNama.add("Fathur");

print(listNama);
}
```

Fungsi

Dalam bahasa pemrograman yang mendukung Object Oriented Programming seperti Dart, fungsi (atau prosedur) memiliki peran penting dalam membangun kode yang terstruktur dan mudah dipelihara. Fungsi digunakan untuk membagi program menjadi bagian-bagian kecil yang memiliki tugas spesifik, sehingga kode menjadi lebih modular dan mudah dipahami.

Untuk menghasilkan kualitas kode yang baik, programmer biasanya menerapkan prinsip-prinsip seperti SOLID, KISS (Keep It Simple, Stupid), dan YAGNI (You Aren't Gonna Need It). Prinsip-prinsip ini menekankan konsep separation of concern, yaitu setiap bagian kode memiliki tanggung jawab yang jelas dan terpisah. Dengan pendekatan ini, kode menjadi lebih rapi, mudah diuji (testable), serta mengurangi penulisan kode berulang (boilerplate code).

Dengan memanfaatkan fungsi secara optimal dan menerapkan prinsip-prinsip tersebut, pengembangan aplikasi menjadi lebih efisien, scalable, dan mudah untuk dikembangkan di masa depan.

Mendefinisikan Fungsi

```
String cekGanjilGenap(int n) {

}
```

Memanggil Fungsi

```
int angka = 8;
String hasil = cekGanjilGenap(angka);
print("Angka " + angka.toString() + " adalah " + hasil);
```

Mengembalikan Nilai

Tambahkan return apabila mendefinisikan sebuah fungsi, contohnya ada pada codingan dibawah yang bisa mengembalikan nilai "Ganjil/Genap" dari angka yang sudah ditentukan.

```
String cekGanjilGenap(int n) {  
    if (n % 2 == 0) {  
        return "Genap";  
    } else {  
        return "Ganjil";  
    }  
}
```

Menambahkan Parameter

Fungsi memiliki scope yang terbatas, tentunya fungsi butuh input dari luar agar program didalamnya bisa memproses tugasnya.

```
String cekGanjilGenap(int n) {  
    if (n % 2 == 0) {  
        return "Genap";  
    } else {  
        return "Ganjil";  
    }  
}  
  
void main() {  
    int angka = 8;  
    String hasil = cekGanjilGenap(angka);  
    print("Angka " + angka.toString() + " adalah " + hasil);  
}
```

Pada fungsi diatas, n merupakan parameter. Variable diluar fungsi yang dibuat agar dapat digunakan didalam fungsi